GrabImage_GpuDirectForVideo_Cuda 示例说明

1. 功能概述

本示例展示了如何使用工业相机 SDK 结合 NVIDIA GPUDirect For Video 技术，实现相机数据的高效采集和 GPU 加速处理。主要功能包括：

- 相机设备枚举与初始化
- 系统内存与 GPU 内存的分配与管理
- 使用 GPUDirect For Video 技术实现零拷贝数据传输
- 多线程分离取流和 GPU 处理，提高性能

2. 技术原理

2.1 GPUDirect For Video 技术

GPUDirect For Video 是 NVIDIA 推出的一项技术，通过 DMA（直接内存访问）直接将相机数据传输到 GPU 内存，实现零拷贝数据传输，大大减少了数据传输的延迟和 CPU 开销。

2.2 DVP (Direct Video Path) API

DVP API 是 NVIDIA 提供的用于管理系统内存和 GPU 内存之间数据传输的接口，它提供了以下功能：

- 创建和管理系统内存与 GPU 内存的 DVP 缓存
- 实现系统内存与 GPU 内存之间的高效数据传输
- 提供同步机制，确保数据传输的正确性

2.3 内存管理

- **系统内存**：使用 `VirtualAlloc` 分配内存， `VirtualLock` 锁定内存，确保内存不被换出到磁盘
- **GPU 内存**：使用 CUDA API 的 `cuMemAlloc` 分配 GPU 内存
- **内存同步**：使用 DVP 同步对象和信号量机制，确保数据传输的正确性

2.4 多线程设计

- **取流线程**（`WorkThread`）：负责从相机获取图像数据并存储到全局帧列表
- **GPU 处理线程**（`GpuThread`）：负责从全局帧列表获取数据并传输到 GPU 内存

- **线程同步**：使用 C++11 的 `std::mutex` 和 `std::lock_guard` 实现线程同步

3. 核心流程

3.1 初始化阶段

1. **初始化 SDK**：调用 `MV_CC_Initialize` 初始化工业相机 SDK
2. **枚举设备**：调用 `MV_CC_EnumDevices` 枚举可用的相机设备
3. **选择设备**：用户选择要使用的相机设备
4. **创建句柄**：调用 `MV_CC_CreateHandle` 创建相机句柄
5. **打开设备**：调用 `MV_CC_OpenDevice` 打开相机设备
6. **设置网络参数**：对于 GigE 相机，设置最佳包大小
7. **获取图像大小**：调用 `MV_CC_GetPayloadSize` 获取图像数据大小
8. **设置进程工作集**：调用 `SetProcessWorkingSet` 为缓存预留足够内存
9. **初始化 CUDA**：调用 `InitCuda` 初始化 CUDA 上下文
10. **分配系统内存**：调用 `AllocateSystemMemoryBuffers` 分配系统内存并创建 DVP 缓存
11. **注册内存到 SDK**：调用 `MV_CC_RegisterBuffer` 将系统内存注册到 SDK
12. **分配 GPU 内存**：调用 `AllocateGpuMemoryBuffer` 分配 GPU 内存并创建 DVP 缓存

3.2 运行阶段

1. **启动取流线程**：创建并启动 `WorkThread` 线程，开始从相机获取图像数据
   取流线程工作：
   - 调用 `MV_CC_StartGrabbing` 开始取流
   - 循环调用 `MV_CC_GetImageBuffer` 获取图像数据
   - 将获取的图像数据存储到全局帧列表
   - 统计取流性能
2. **启动 GPU 处理线程**：创建并启动 `GpuThread` 线程，开始处理图像数据
   GPU 处理线程工作：
   - 从全局帧列表获取图像数据
   - 调用 `RunSysmemToGPU` 将数据传输到 GPU 内存
   - 统计 GPU 处理性能
   - 调用 `MV_CC_FreeImageBuffer` 释放图像缓冲区

3.3 结束阶段

1. **停止取流**：设置 `g_bExit` 标志为 true，通知线程退出
2. **等待线程结束**：调用 `join` 等待取流线程和 GPU 处理线程结束
3. **关闭设备**：调用 `MV_CC_CloseDevice` 关闭相机设备
4. **销毁句柄**：调用 `MV_CC_DestroyHandle` 销毁相机句柄
5. **反初始化 SDK**：调用 `MV_CC_Finalize` 反初始化 SDK

6. **释放系统内存**：调用 `FreeSystemMemoryBuffers` 释放系统内存和 DVP 缓存
7. **释放 GPU 内存**：调用 `FreeGpuMemoryBuffer` 释放 GPU 内存和 DVP 缓存
8. **反初始化 CUDA**：调用 `DeinitCuda` 反初始化 CUDA 上下文

4. 编译与运行

4.1 环境要求

- 工业相机 SDK
- Visual Studio 2013 或更高版本
- CUDA Toolkit
- NVIDIA GPU Direct for Video SDK
- 需要支持 GPUDirect For Video 技术的 NVIDIA GPU， Supported GPUs：
  - [RTX A6000, A5500, A5000, A4500, A4000]
  - [Quadro RTX 8000, 6000, 5000, 4000]
  - [Quadro GV100, GP100, P6000, P5000, P4000]
  - [Quadro M6000, M5000, M4000]
  - [Quadro K4000, K4200, K5000, K5200 and K6000]
  - [Quadro 4000, 5000, and 6000]
  - [Quadro M5000M equipped mobile workstations]
  - [Quadro K5100M equipped mobile workstations]
  - [A2, A10, A30, A16, A40, A100]
  - [GRID K1 and K2]
  - [Tesla T4]
  - [Tesla V100, P100, K10, K20, K20X, K40]
  - [Tesla C2075 and M2070Q]

4.2 运行步骤

1. 使用 Visual Studio 打开项目 `GrabImage_GpuDirectForVideo_Cuda.sln`
2. 编译项目
3. 运行可执行文件
4. 选择要使用的相机设备
5. 程序开始取流和 GPU 处理
6. 按任意键停止取流，显示性能统计信息
7. 程序释放资源并退出